

Skripta za ispit iz predmeta Razvoj zasnovan na komponentama (RZK)

Skripta je organizovana prema zvaničnim ispitnim pitanjima. Na ispitu se izvlači po jedno pitanje iz svake od dve grupe.

I GRUPA PITANJA

1. Šta je to Razvoj zasnovan na komponentama?

Osnovna ideja

Razvoj zasnovan na komponentama (RZK, Component-Based Software Development) teži izgradnji softverskih sistema na osnovu već izgrađenih softverskih jedinica – **komponenti**.

- Podrazumeva rešavanje problema njegovom **dekompozicijom na manje celine**.
- Predstavlja **bottom-up pristup** – gotove komponente su polazna tačka za izgradnju sistema.
- Sistem se izgrađuje **sklapanjem delova**, umesto razvoja jednog monolitnog entiteta.
- Svaki softverski sistem poseduje elemente koji se mogu ponovo iskoristiti.
- Kompanije teže pojednostavljenom i ubrzanom razvoju softvera.

Istorijat

Pojam RZK-a se prvi put pojavljuje 60-ih godina prošlog veka – softverskom komponentom se smatrala masovno proizvedena softverska jedinica koja se može više puta koristiti (npr. kompajleri za porodicu jezika). Sveprisutnost kompleksnih sistema velikih razmera povećava potrebu za rešenjima baziranim na komponentama.

Osnovni elementi softverskog inženjeringa zasnovanog na komponentama

1. Nezavisne komponente, specificirane putem interfejsa
2. Standardi za lakšu integraciju
3. Middleware za podršku interoperabilnosti (komponentni modeli)
4. Ponovno iskoristiv proces razvoja

Prednosti RZK-a

- Smanjeni troškovi proizvodnje
- Skraćeno vreme razvoja
- Povećane mogućnosti ponovne upotrebe

Životni ciklus komponenti u okviru RZK-a (3 faze – idealizovan ciklus, razlikuje se od životnog ciklusa sistema):

1. **Faza dizajna** – dizajn, definicija i konstrukcija komponenti
2. **Faza razmeštanja** – deploy-ovanje komponenti unutar sistema koji se razvija
3. **Run-time faza** – instanciranje komponenti u run-time-u

Ciljevi RZK-a

- **Savladavanje složenosti** – tehnike: apstrakcija, dekompozicija, inkrementalni razvoj
 - **Skraćivanje vremena isporuke**
 - **Poboljšanje konzistentnosti među sistemima** – deljenje karakteristika smanjuje troškove
 - **Upotreba najboljeg raspoloživog softvera**
 - **Povećanje produktivnosti**
 - **Poboljšanje kvaliteta** – korektan sistem odmah, bez dugog testiranja i popravljanja
 - **Povećanje vidljivosti napretka projekta** – zahteva inkrementalni pristup
 - **Podrška za paralelni i distribuirani razvoj**
 - **Smanjivanje troškova održavanja** – laka identifikacija potrebe za izmenom i procena uticaja
-

2. Šta su komponente?

Ne postoji jedinstvena definicija komponenti. Najšire prihvaćena definicija (Szyperski, Gruntz, Murer, 2002):

Softverska komponenta je jedinica kompozicije sa ugovorno specificiranim interfejsima i eksplicitnim, isključivo kontekstnim zavisnostima. Softverska komponenta može se nezavisno otpremati i koristiti za kompoziciju od strane trećih lica.

- Komponente se koriste kao osnove dizajna, implementacije i održavanja sistema.
- Komponenta je **nezavisno isporučiv segment funkcionalnosti** koji pruža pristup svojim servisima kroz interfejse.

Komponente i objekti

- Komponenta je kolekcija objekata pomoću kojih obezbeđuje neke servise.
- Komponenta je apstraktniji pojam – klase su previše detaljne/specifične da bi predstavljale komponente.
- Komponenta može postojati samostalno.
- Komponenta je jedinica isporuke (**unit of deployment**), bazirana na komponentnom modelu.
- Komponenta omogućava paketiranje jedne ili više objektnih implementacija.
- Komponenta je jedinica sklapanja (**unit of assembly**) za dizajn i konstrukciju sistema od većeg broja nezavisnih funkcionalnih celina.

Model komponente

Definiše i komponente i mehanizme kompozicije. Obuhvata:

- **Semantiku** komponenti – šta komponente treba da predstavljaju
- **Sintaksu** komponenti – kako se komponente definišu, konstruišu i reprezentuju
- **Kompoziciju** komponenti – kako se komponente sklapaju

Generalno prihvaćen pogled: komponenta je softverska jedinica sa servisima koje pruža i koji su joj potrebni.

Elementi komponente

- **Specifikacija** – apstraktni opis servisa koje nudi
 - **Implementacija** – jedna ili više
 - **Komponentni standard** – komponentni model, skup servisa koji podržavaju rad komponenti zajedno sa skupom pravila koje moraju zadovoljiti
 - **Pristup pakovanju** – grupisanje skupa servisa
 - **Pristup razmeštaju** – kreiranje izvršive instance komponente i omogućavanje interakcije sa njom
-

3. Šta je to SOA i kakav je odnos između SOA i mikroservisa?

Servisno orijentisana arhitektura (SOA)

SOA predstavlja pristup razvoju softvera koji koristi softverske komponente, tzv. **servise**, u cilju kreiranja poslovnih aplikacija.

- Svaki servis pruža određene funkcionalnosti.
- Servisi mogu međusobno komunicirati, bez obzira na platforme i jezike.
- SOA omogućuje ponovnu upotrebu servisa u različitim sistemima, kao i njihovo kombinovanje radi izvršavanja složenih zadataka.

Prednosti SOA-e

- Kraće vreme razvoja softvera (upotreba postojećih servisa)
- Jednostavnije održavanje pojedinačnih servisa kao manjih celina (ali održavanje celine može biti izazovno)
- Lakše prilagođavanje novim tehnologijama

Osnovni principi SOA-e

1. **Interoperabilnost** – bilo koji klijent može pozvati servis, bez obzira na tehnologiju implementacije

2. **Slabo povezivanje (loose coupling)** – servisi ne čuvaju prethodna stanja, modifikacija servisa ne sme značajno uticati na klijente
3. **Apstrakcija** – korisnik ne zna detalje implementacije, servis je crna kutija
4. **Granularnost** – servisi treba da budu odgovarajuće veličine i opsega

Osnovni elementi SOA-e

- **Servisi** – osnovne gradivne jedinice (implementacija, ugovor, interfejs)
- **Servis provajder** – kreira, održava i pruža mogućnost upotrebe servisa
- **Korisnik servisa (service consumer)** – drugi servis ili aplikacija
- **Registar servisa (service registry)** – repozitorijum koji nudi pristup dostupnim servisima preko mreže

Komunikacioni protokoli SOA-e: JMS (Java Message Service), Apache ActiveMQ, SOAP, RESTful HTTP.

Izazovi SOA-e

- Ograničena skalabilnost (servisi dele mnogo resursa)
- Povećanje međuzavisnosti sa rastom složenosti sistema
- Postojanje jedinstvene tačke greške (ESB – Enterprise Service Bus, centralizovan servis koji povezuje servise i klijente)

Mikroservisi u odnosu na SOA

- Mikroservisi predstavljaju **evoluciju SOA arhitekture** – prevazilaze njene nedostatke (npr. centralizovanost ESB-a).
- Osnovna ideja mikroservisa: saradnja slabo povezanih procesa; svaki mikroservis se fokusira na jedan specifičan zadatak.
- Mikroservisi omogućuju pristup preko API-ja koje sami definišu, **umesto preko ESB-a**.

Osobine mikroservisa

- Nezavisno skalabilni (horizontalna skalabilnost), brzi, portabilni, nezavisni od platforme, pogodni za cloud
- Samoodrživost i fleksibilnost
- Slaba povezanost – minimalna zavisnost od drugih mikroservisa
- Fina granulacija, sa preferencijom dupliciranja podataka radi lokalne dostupnosti

Izazovi mikroservisa: kompleksnost i povezivanje mikroservisa, potreba za posebnim veštinama, distribuirana bezbednost i testiranje.

Mikroservisi i kontejneri: Mikroservisna arhitektura je veoma pogodna cloud okruženjima, najčešće funkcionišu unutar kontejnera (npr. Docker). Orkestrator (npr. Kubernetes) omogućuje upravljanje kontejnerima.

Da li su mikroservisi uvek rešenje? Ne – i monolitna i mikroservisna arhitektura imaju prednosti. Kriterijumi za odluku:

- Kompleksnost sistema – mikroservisi isplativiji za veće, složenije sisteme
 - Veličina i veštine razvojnog tima
 - Očekivani rast sistema
 - Troškovi razvoja – monolit brži za inicijalni razvoj, mikroservisi se kasnije lakše skaliraju i prilagođavaju
-

4. Web servisi

Šta su web servisi?

Naivna definicija: "Web servis je servis koji se isporučuje preko web-a" – nije potpuna, jer nije svaka aplikacija koja pruža uslugu preko weba ujedno i web servis.

Motivacija: Ako druga aplikacija treba da koristi neke funkcionalnosti naše aplikacije – kreiranje JAR fajla bi značilo da ta aplikacija mora imati pristup istoj bazi i svim zavisnostima, što otežava izmene i ne podržava interoperabilnost.

Prava definicija:

Web servis je softverski sistem, dizajniran tako da podržava interoperabilnu interakciju od-mašine-do-mašine putem mreže.

Osobine web servisa

Tri ključne osobine:

1. Dizajnirani su za interakciju između aplikacija (mašina)
2. Potrebno je da budu interoperabilni, nezavisni od platforme
3. Potrebno je da mogu komunicirati putem mreže

Request i Response: osnovni koncepti iza web servisa. Aplikacija koja konzumira servis šalje **request**, servis vraća **response**. Oba treba da budu nezavisna od platforme, u formatu koji sve platforme podržavaju:

- **XML** – eXtensible Markup Language
- **JSON** – JavaScript Object Notation

Format za request/response definiše se putem **definicije servisa (service definition)**, koja sadrži:

- Definisan format za request/response
- Definisanu strukturu request-a
- Definisanu strukturu response-a

- **Endpoint** – gde je servis dostupan

Terminologija

Pojam	Značenje
Request	zahtev, ulaz prilikom poziva servisa
Response	odgovor, izlaz prilikom poziva servisa
Message exchange format	format za request/response
Servis provajder / server	onaj koji host-uje servis
Definicija servisa	ugovor između provajdera i korisnika
Endpoint	URL na kome je servis dostupan
Transport	način poziva servisa (HTTP putem URL-a, ili MQ)

Upotreba Message Queue-a: korisnik servisa stavlja poruku u queue, servis provajder je uzima i vraća odgovor kroz queue; sva komunikacija ide preko MQ-a.

Tipovi web servisa

Dve grupe:

1. **Bazirani na SOAP-u**
2. **REST servisi**

Napomena: SOAP i REST nisu direktno uporedivi – REST definiše arhitekturni pristup (stil), dok SOAP postavlja ograničenja na format XML-a koji se razmenjuje.

SOAP

- Nekada skraćenica od Simple Object Access Protocol (danas se to više ne koristi kao pun naziv).
- Predstavlja DOM (Document Object Model), XML je format razmene.

Elementi strukture SOAP poruke:

- **SOAP envelope** – spoljni element XML poruke
- **SOAP header** – zaglavlje sa meta-podacima (autentifikacija, autorizacija...), opciono
- **SOAP body** – sadržaj request-a/response-a

Podržan je transport preko MQ-a i preko HTTP-a. Definicija servisa: **WSDL** (Web Service Definition Language) – definiše endpoint, sve dostupne operacije, strukturu request-a i

response-a.

Prednosti SOAP-a: proširivost (XML), podrška raznih protokola (HTTP, SMTP, MSMQ), jaka tipiziranost (XML šema), standardizovano upravljanje greškama (fault element), ugrađena bezbednost.

Mane SOAP-a: lošije performanse (velike poruke), kompleksnost, manja intuitivnost u odnosu na REST. Danas se smatra zastarelim pristupom.

RESTful web servisi

- REST = **REpresentational State Transfer** (Roy Fielding); "ful" dodato radi promovisanja upotrebe tehnologije.
- Cilj REST-a: što bolje iskoristiti HTTP protokol.

HTTP protokol definiše zaglavlje i telo poruke:

- **General header** – primenjivo i na request i response (npr. cache control)
- **Request header** – samo za request (npr. accept)
- **Response header** – samo za response
- **Entity header** – meta informacije o telu poruke

Resurs – najvažnija apstrakcija u REST-u:

- Predstavlja sve što aplikacija nudi spoljnom svetu.
- Svaki resurs ima **URI** (Uniform Resource Identifier).
- Aplikacija sadrži listu resursa.
- REST ne propisuje format reprezentacije resursa (XML, HTML, JSON).
- REST omogućuje da se resursi definišu i da se nad njima vrše akcije korišćenjem koncepata koje nudi HTTP (GET, POST, PUT, DELETE...).

REST ne ograničava format razmene podataka, ali je **JSON** popularan. Ne postoji standardni oblik definicije servisa, ali postoje pokušaji:

- **WADL** (Web Application Definition Language) – nije postigao veliku popularnost
- **Swagger / OpenAPI** – popularan pristup

RESTful vs SOAP

Aspekt	SOAP	REST
Format razmene	uvek XML, striktna struktura	fleksibilan, najčešće JSON
Definicija servisa	WSDL	nema standardan (Swagger/OpenAPI)
Transport	bez ograničenja	striktno HTTP

Aspekt	SOAP	REST
Jednostavnost	kompleksniji (XML parsiranje)	jednostavniji (JSON)

5. RESTful web servisi (kreiranje RESTful API-ja u Spring Boot-u)

(Nastavak na prethodno pitanje – fokus na praktičnu implementaciju)

- Spring Boot je jedan od najkorišćenijih framework-a za izgradnju REST API-ja.
- Na <https://start.spring.io/> moguće je generisati proizvoljan Spring Boot projekat (Spring Initializr).
- Preporuka: koristiti najnoviju stabilnu verziju, izbegavati SNAPSHOT verzije.

Anotacije

- `@RestController` – kombinacija `@Controller` i `@ResponseBody`; svaka metoda koja obrađuje request automatski serijalizuje povratni objekat u HTTP response.
- `@RequestMapping` – mapiranje request-a na metode kontrolera, zahteva specifikaciju tipa request metode.
- Od Spring 4.3: `@GetMapping`, `@PostMapping`, `@PutMapping`, `@DeleteMapping`, `@PatchMapping`.

Obrada request-a

- **DispatcherServlet (DS)** – mapira se na root URL (`/`), mapira servlete i usmerava request na odgovarajući kontroler i metodu.
- Konfiguriše se automatski (`DispatcherServletAutoConfiguration`) – na osnovu klase na classpath-u, Spring Boot detektuje da se radi o web/REST aplikaciji.

Formiranje response-a

- `@ResponseBody` – vraća bean "onakav kakav jeste"; podrazumevana konverzija koristi `JacksonHttpMessageConverters` (rezultat auto-konfiguracije).

Mapiranje grešaka

- Pogrešan URL → **Whitelabel Error Page**, status 404 (rezultat Auto Configuration-a).
- Sve potrebne biblioteke navedene su u `pom.xml` (`dependencies`), biraju se prilikom kreiranja Starter projekta.

Path varijable

- `@PathVariable` – vezuje parametar metode za URI template varijablu.


```
java
```

```
@GetMapping("/users/{id_user}")  
public User retrieveUser(@PathVariable("id_user") Integer idUser)
```

Pozivanje REST servisa / klijenti

- Ukucavanjem URL-a u browser automatski se poziva GET metoda.
- Za druge tipove metoda potreban je REST klijent: **Postman**, **Insomnia**, ili browser ekstenzije (npr. Talend API za Chrome).

HTTP statusni kodovi

5 klasa: Informational (100-199), Successful (200-299), Redirection (300-399), Client error (400-499), Server error (500-599).

Najčešće korišćeni:

Kod	Značenje
200	Success
201	Created
204	No content
400	Bad Request
401	Unauthorized
404	Resource is not found
500	Server exception

Mapiranje request-a i response-a

- `@RequestBody` - mapira telo HTTP request-a na domenski objekat (automatska deserijalizacija u Java objekat).
- `@ResponseBody` - automatska serijalizacija Java objekta u JSON, u telo HTTP response-a.
- `ResponseEntity` - proširenje `HttpEntity`, dodaje i statusni kod.

Struktura aplikacije

Primena **Controller - Service - Repository** paterna.

Upravljanje izuzecima

Spring nudi nekoliko opcija:

1. `@ExceptionHandler` na nivou kontrolera - upravljanje izuzecima samo unutar tog kontrolera.

```
java
```

```
@ExceptionHandler({ CustomException1.class, CustomException2.class })  
public void handleException()
```

2. `HandlerExceptionResolver`

- `ExceptionHandlerExceptionResolver` (od Spring 3.1)
- `DefaultHandlerExceptionResolver` (od Spring 3.0) - razrešava standardne Spring izuzetke na HTTP statusne kodove, bez postavljanja tela response-a
- `ResponseStatusExceptionResolver` (od Spring 3.0) - koristi `@ResponseStatus` anotaciju na custom izuzecima
- Custom `HandlerExceptionResolver` - kombinacija prethodna dva, moćan pristup, ali bez kontrole nad telom response-a

3. `@ControllerAdvice` (od Spring 3.2) - globalni `@ExceptionHandler`; konsolidacija različitih handlera u jednu komponentu, puna kontrola nad telom response-a i statusnim kodom, oslanja se na `ResponseEntity`.

4. `ResponseStatusException` (od Spring 5) - kreira se instanca pomoću `HttpStatus`-a i opciono uzroka.

Primer - custom izuzetak

```
java
```

```
throw new UserNotFoundException("User with id: " + idUser + " not found");
```

```
java
```

```
@ResponseStatus(code = HttpStatus.NOT_FOUND)  
public class UserNotFoundException extends RuntimeException {  
    public UserNotFoundException(String msg) {  
        super(msg);  
    }  
}
```

Generičko upravljanje izuzecima za sve resurse

- Kreira se klasa entiteta greške (npr. `ErrorDetail`) sa poljima: datum, poruka, detalji.
- Klasa nasleđuje `ResponseEntityExceptionHandler` (sadrži `@ExceptionHandler` metodu koja obrađuje sve izuzetke Spring MVC-a) i anotira se sa `@ControllerAdvice`.

```
java

@ExceptionHandler(UserNotFoundException.class)
public final ResponseEntity<ErrorDetail> handleNotFoundExceptions(
    Exception ex, HttpServletRequest request) throws Exception {
    ErrorDetail ed = new ErrorDetail(LocalDate.now(),
        ex.getMessage(), request.getDescription(false));
    return new ResponseEntity<>(ed, HttpStatus.NOT_FOUND);
}

@ExceptionHandler(Exception.class)
public final ResponseEntity<ErrorDetail> handleAllExceptions(
    Exception ex, HttpServletRequest request) throws Exception {
    ErrorDetail ed = new ErrorDetail(LocalDate.now(),
        ex.getMessage(), request.getDescription(false));
    return new ResponseEntity<>(ed, HttpStatus.INTERNAL_SERVER_ERROR);
}
```

Validacija

- Validacija ulaza je neophodna da bi se obradili neželjeni slučajevi (npr. korisnik sa praznim imenom ne treba da se snimi u bazu).
- Zavisnost: `spring-boot-starter-validation`.

Validacija bean-ova - anotacije nad poljima:

Anotacija	Značenje
<code>@NotNull</code>	polje ne sme biti null
<code>@NotBlank</code>	string ne sme biti prazan, mora imati bar jedan karakter
<code>@Min</code> , <code>@Max</code>	numeričko polje mora biti u zadatom opsegu
<code>@Pattern</code>	string mora zadovoljiti regularni izraz
<code>@Email</code>	mora biti validna email adresa
<code>@Past</code>	datum mora biti iz prošlosti
<code>@Size</code>	ograničenje broja karaktera

@Valid

- Koristi se na parametrima metoda – osigurava validaciju parametra.
- PUT/POST metode primaju objekte kroz `@RequestBody`, koji se automatski prevode u Java objekte; sa `@Valid` to prevođenje podleže validaciji.
- Može se koristiti i za anotiranje polja (ugnježdeni kompleksni tipovi).
- Neuspešna validacija → `MethodArgumentNotValidException` → status **400 Bad Request** (podrazumevano).

@Validated

- Validacija request parametara i path varijabli – kontroler se anotira sa `@Validated`.
- Neuspešna validacija → `ConstraintViolationException` (Spring ne registruje default handler za ovo).

Grupe validacije

- Isti bean se koristi za različite REST servise (npr. kreiranje i update korisnika); potrebna je validacija koja zavisi od slučaja korišćenja.
 - Anotacijama se dodaje oznaka grupe: `@Null(groups = OnCreate.class)`, `@NotNull(groups = OnUpdate.class)`.
 - `OnCreate`/`OnUpdate` mogu biti marker interfejsi.
 - Metode se anotiraju: `@Validated(OnCreate.class)`; klasa kontrolera takođe mora imati `@Validated`.
 - Grupe validacije treba koristiti oprezno.
-

7. Dokumentacija, Content negotiation i Internacionalizacija u Spring Boot-u

Dokumentacija

Korisnici REST API-ja moraju razumeti: resurse, akcije, strukturu request-a/response-a (uključujući ograničenja i validacije).

Izazovi: tačnost (up-to-date), konzistentnost (posebno kod velikog broja API-ja).

Dva pristupa:

1. Ručno održavanje dokumentacije – veliki napor, sklono propustima
2. Generisanje dokumentacije iz koda

Swagger i OpenAPI

- 2011. – uvedena Swagger specifikacija i alati.

- 2016. - kreirana **Open API** specifikacija, bazirana na Swagger specifikaciji.
- OpenAPI je standardan, jezički-nezavistan interfejs koji pomaže razumevanju REST API-ja.
- **Swagger UI** - omogućuje vizualizaciju i interakciju sa REST API-jem.

OpenAPI opisuje: endpoint-e i operacije nad njima (GET, POST...), ulazne/izlazne parametre operacija, metode autentifikacije, kontakt informacije, licence, pravila korišćenja.

Swagger alati:

- **Swagger Editor** - browser editor za pisanje OpenAPI definicija
- **Swagger UI** - renderuje OpenAPI definicije kao interaktivnu dokumentaciju
- **Swagger Core** - biblioteke za kreiranje/konzumiranje OpenAPI definicija
- **Swagger Parser** - biblioteka za parsiranje

Praktično: dodaje se zavisnost (`spring-openapi-starter-webmvc-ui`); dokumentacija dostupna na (`localhost:8080/swagger-ui/index.html`).

Content negotiation (pregovaranje po pitanju sadržaja)

- Isti URI ponekad treba da vrati različite reprezentacije (JSON, XML) ili jezik sadržaja.
- Potreban je način da consumer kaže provider-u šta mu je potrebno - ovo omogućuje **content negotiation**.

Primer: API treba da vraća JSON ili XML u zavisnosti od potreba consumer-a.

- Dodaje se zavisnost (`jackson-dataformat-xml`).
- Klijent u pozivu dodaje header (`Accept: application/xml`).
- Aplikacija vraća XML reprezentaciju umesto JSON-a.

Internacionalizacija (i18n)

- Skraćeno **i18n** - 18 karaktera između "i" i "n" u reči "internationalization".
 - Tipično se koristi HTTP request header (`Accept-Language`) da bi consumer specificirao željeni jezik/locale.
 - Standardna implementacija:
 - Kreira se (`messages.properties`) fajl u resursima sa prevodima (npr. `good.morning.message=Good morning`).
 - Injektuje se (`MessageSource`) objekat i poziva (`getMessage("good.morning.message", null, "Default message", locale)`).
-

8. Verzionisanje, Hateoas, Filteri

Verzionisanje

Strategija verzionisanja omogućuje korisnicima da koriste postojeći API i migriraju na noviju verziju kada su spremni – neophodno kada se uvode bitne izmene koje utiču na korisnike.

4 uobičajene strategije:

1. **Drugačiji URL** (`/v2/users`) – nije u duhu REST principa (verzija ne bi trebalo da bude u URI-ju); koristi npr. Twitter
2. **Request parametri** – query parametar sa verzijom, kviri čitljivost URL-a, meša se sa parametrima endpointa; koristi npr. Amazon
3. **Custom Request Headers** – URL ostaje isti, verzija u header-u (npr. `X_API_VERSION=1`), nije lako vidljivo; koristi npr. Microsoft
4. **Media type** – po principu content negotiation, nije toliko user-friendly; koristi npr. GitHub

Faktori za razmatranje pri izboru pristupa:

- Zagađivanje URI-ja (URL i request parametri)
- Zloupotreba HTTP zaglavlja (custom header i media type)
- Keširanje – radi se na osnovu URL-a, kod header-a/media type-a keširanje je otežano
- Mogućnost izvršavanja iz browser-a – kod header/media type pristupa nije moguće bez klijenta
- Dokumentacija API-ja se obično ne generiše na osnovu header-a

Ne postoji idealno rešenje – preporuka je konzistentnost na nivou kompanije.

HATEOAS

Hypermedia As The Engine Of Application State (HATEOAS)

- REST API vraća podatke i, uz to, pristup narednim mogućim akcijama (novim REST pozivima).

Primer response-a sa linkovima:

```
json
```

```
{
  "ime": "Pera",
  "datumRodjenja": "2000-11-12",
  "_links": {
    "svi_korisnici": {
      "href": "http://localhost:8080/users"
    }
  }
}
```

Implementacija:

1. Kreiranje specijalne forme bean-a - svi podaci u bean; teško za održavanje.
2. Korišćenje standardne implementacije - automatizovan proces:
 - **HAL** (JSON Hypertext Application Language) - jednostavan, konzistentan format za linkove među resursima
 - **Spring HATEOAS** - generiše HAL odgovore sa linkovima ka resursima

Filtriranje

- Prilikom serijalizacije objekat se konvertuje u stream (npr. JSON); najpopularniji alat za serijalizaciju u Javi je **Jackson**.
- Standardno ponašanje: polja Java objekta se mapiraju na istoimena polja u JSON-u; menja se pomoću `@JsonProperty("user_name")`.
- Filtriranje - potreba da se neka polja izostave (npr. `password`).

Dva tipa filtriranja:

1. **Statičko filtriranje** - isto filtriranje na nivou bean-a za sve REST API-je.
 - `@JsonIgnore` - nad poljem klase, izostavlja polje iz JSON reprezentacije; nije osetljiv na promenu naziva polja.
 - `@JsonIgnoreProperties` - nad klasom, navodi polja koja se izostavljaju; osetljiv je na promenu naziva polja.
2. **Dinamičko filtriranje** - specifično filtriranje za različite REST API-je, logika je na nivou REST API-ja.
 - `@JsonFilter("nazivFiltera")` na klasi bean-a.
 - REST API vraća instancu `MappingJacksonValue` (sadrži instancu bean-a), na kojoj se poziva `setFilters(filters)`.

java

```
SimpleBeanPropertyFilter f = SimpleBeanPropertyFilter
    .filterOutAllExcept("field1", "field3");
FilterProvider fp = new SimpleFilterProvider()
    .addFilter("nazivFiltera", f);
```

9. Monitoring, Bezbednost

Monitoring

- Monitoring endpoint-a REST API-ja - praksa nadgledanja i evaluacije dostupnosti, performansi i funkcionalnosti API endpoint-ova.
- Spring Boot Actuator** - alat za monitoring, obezbeđuje "production ready" osobine, omogućuje nadgledanje i upravljanje aplikacijom u produkciji. Dodaje se kao zavisnost (`spring-boot-starter-actuator`).

Neki endpoint-i Actuator-a:

Endpoint	Opis
<code>beans</code>	kompletan spisak bean-ova u aplikaciji
<code>health</code>	informacije o stanju aplikacije
<code>metrics</code>	metrike aplikacije
<code>mappings</code>	detalji o RequestMappings

- Pristup: `localhost:8080/actuator`.
- Za sveobuhvatne informacije: `management.endpoints.web.exposure.include=*`.
- Spring Boot HAL Explorer** - API explorer za RESTful Hypermedia API-je (upotrebom HAL-a); omogućuje i ne-tehničkom delu tima da eksperimentiše sa API-jem. Pristup preko `localhost:8080`, dok `/actuator` daje pristup Actuator-u kroz explorer.

Bezbednost

- Spring Security** - moćan i fleksibilan framework za autentifikaciju i kontrolu pristupa.

Osnovna primena:

- Dodavanjem zavisnosti `spring-boot-starter-security`, pri pokretanju aplikacije generiše se password ("Using generated security password...").

- REST poziv iz browser-a preusmerava na Login stranicu; poziv preko REST klijenta (npr. Talend API) vraća status **401**.
- Podrazumevano se može ulogovati kao `user` sa generisanom šifrom.
- U `application.properties` moguće je podesiti `spring.security.user.name` i `spring.security.user.password`.
- Sa dodatim Spring Security-jem, POST zahtevi vraćaju status **403 - Forbidden**.

Razlog: svaki request se presreće od strane Spring Security-ja i izvršava se serija tzv. **Filter chain-a**:

- Svi zahtevi treba da se autentifikuju.
- Ako zahtev nije autentifikovan, prikazuje se login stranica.
- Zahteva se **CSRF token** (Cross-Site Request Forgery) za POST i PUT zahteve.

Filter chain je moguće prilagoditi kreiranjem klase sa `@Configuration` anotacijom, unutar koje se podešavaju koraci filter chain-a.

II GRUPA PITANJA

10. Mikroservisi u Spring Boot-u

Monolitna aplikacija

- Velika aplikacija sa centralizovanom bazom podataka. Uvođenje nove verzije i održavanje su zahtevni zadaci.

Izazovi monolitne arhitekture:

- Kompleksno deploy-ovanje (i za male izmene potrebno je redeploy-ovati celu aplikaciju)
- Usko povezane (tightly coupled) komponente - izmene u jednom delu utiču na druge, produžen ciklus release-a
- Ograničena skalabilnost - skaliranje zahteva resurse čak i kada samo jedan deo aplikacije treba veći kapacitet
- Tehnološka zaključanost - ceo sistem je u istoj tehnologiji

Mikroservisi - definicija

┃ Mikroservisi su mali, autonomni servisi koji rade zajedno. - Sam Newman

- Iako su nezavisne jedinice, postoji minimalna količina centralizovanog upravljanja.
- Grade se u skladu sa zahtevima poslovanja i nezavisno se deploy-uju automatizovanim mehanizmima.

- Mikroservisni arhitekturni stil = razvoj jedne aplikacije kao kolekcije malih servisa, gde se svaki izvršava u posebnom procesu i komunicira "lightweight" mehanizmima.

Najvažnije osobine mikroservisa:

- Dostupni su putem REST-a
- Male, dobro definisane jedinice koje se deploy-uju nezavisno
- Omogućuju dinamičko skaliranje, nezavisno jedni od drugih
- Pogodni za cloud (cloud enabled) - moguće je imati višestruke instance jednog servisa; lako se dodaju/uklanjaju instance u zavisnosti od opterećenja

Primer arhitekture (sistem za rad sa filmovima): Movie, Booking, Pricing, Customer, Review mikroservisi.

Izazovi pri radu sa mikroservisima:

- Kako odrediti granice pojedinačnih mikroservisa (šta svaki treba da radi) - proces koji evoluirao
 - Upravljanje konfiguracijom - sinhronizacija rada skupa mikroservisa
 - Dinamičko skaliranje (dynamic load balancing)
 - Vidljivost - monitoring i pronalaženje grešaka
 - Tolerancija na greške (fault tolerance) - loš dizajn dovodi do "efekta kule od karata"
-

11. Spring Cloud

- **Spring Cloud** (spring.io/projects/spring-cloud) obezbeđuje alate za izgradnju distribuiranih sistema; predstavlja skup različitih projekata.
- Primer: **Spring Cloud Netflix** - Netflix je rano počeo sa mikroservisnom arhitekturom i open-source-ovao mnoštvo komponenti pod ovim projektom.

Kako Spring Cloud rešava izazove mikroservisne arhitekture

Upravljanje konfiguracijom

- Preko **Spring Cloud Config servera** - čuvanje konfiguracije za različita okruženja, za različite mikroservise, u Git repozitorijumu.

Dinamičko skaliranje

- Naming server **Eureka**, load balancing preko Eureka, **Spring Cloud Load Balancer**-a i **Feign**-a (jednostavni REST klijenti).
- Postoji niz instanci jednog mikroservisa, opterećenje se raspoređuje među njima; svi mikroservisi se registruju putem naming servera.

Naming server - dve značajne osobine:

1. Mogućnost registracije servisa
2. Pronalaženje servisa (**service discovery**) – pristup URL-ovima, dobijanje instance potrebnog servisa; olakšava dinamičku vezu među servisima

Vidljivost i monitoring

- **Zipkin** – distributed tracing server za praćenje request-a kroz komponente.
- **Spring Cloud Gateway** – zajedničke osobine mikroservisa (logging, security, analitika) se ne implementiraju ponovo za svaki servis, API gateway daje rešenje.

Tolerancija na greške

- Konfigurisanje ponašanja pri pojavi greške – **Resilience4j**.

Važni koncepti Spring Cloud-a: Konfiguracija, Load balancing, Service discovery, Distributed tracing, Edge Server, Fault tolerance.

Prednosti mikroservisne arhitekture

- Adaptacija novih tehnologija i procesa je pojednostavljena – mikroservisi komuniciraju jednostavnim porukama, mogu biti izgrađeni različitim tehnologijama.
- Dinamičko skaliranje – utiče i na hardver, aplikacija se skalira prema trenutnim potrebama.
- Nove osobine brže stižu do produkcije.

12. Spring Cloud Config

- Omogućuje podršku sa klijentske i serverske strane za **eksternalizovanu konfiguraciju** distribuiranih sistema.
- Obezbeđuje **centralizovano mesto** za upravljanje osobinama aplikacija.
- Oslanja se na upotrebu **Git**-a.
- Konfiguracija se može menjati tokom runtime-a.

Pod konfiguracijom se podrazumevaju podešavanja koja se razlikuju po okruženjima (DEV, TEST, PROD) – npr. kredencijali za bazu, URL-ovi eksternih API-ja, i ostala specifična konfiguracija.

Spring Cloud Config Server i Client

- **Spring Cloud Config Server** – povezuje se sa Git repozitorijumom, odakle čita potrebnu konfiguraciju.
- **Spring Cloud Config Client** – klijenti se povezuju sa Config Server-om odakle dobijaju konfiguracione detalje.

Spring Cloud Config Server - detalji

- Kreira se kao Spring Boot projekat sa zavisnošću za Spring Cloud Config Server.
- Unutar Git repozitorijuma smeštaju se `.properties` fajlovi za razne klijente (mikroservise).
- U `application.properties` fajlu servera specificira se lokacija Git repozitorijuma.
- Glavna klasa se anotira sa `@EnableConfigServer`.
- Moguće je čuvati konfiguraciju za različita okruženja različitih aplikacija - npr. `myservice-dev.properties`, `myservice-prod.properties`.

Spring Cloud Config Client - detalji

- Kreira se kao Spring Boot projekat sa zavisnošću za Spring Cloud Config Client.
- Po default-u, Spring Boot aplikacija čita konfiguraciju iz lokalnog `application.properties`.
- Za promenu ovog ponašanja: fajl se preimenuje u `bootstrap.properties` i unutar njega se specificira URL config servera.
- U `bootstrap.properties` moguće je specificirati koje je okruženje aktivno (može se čitati i kao argument).

13. Povezivanje mikroservisa

Postoje razni načini za pozivanje REST servisa:

- Biblioteke za kreiranje REST klijenata - **Jersey**, **REST Template...**
- **Open Feign** klijent
- Korišćenje **Middleware infrastrukture** - Kafka, RabbitMQ...

REST Template

- Sinhrona varijanta klijenta za izvršavanje HTTP zahteva.
- Omogućuje transformisanje request-a/response-a iz JSON/XML u Java objekte.
- `getForEntity()` - izvršava GET request i vraća `ResponseEntity` (statusni kod + resurs).

```
java
```

```
ResponseEntity<MyEntity> responseEntity = new RestTemplate()  
    .getForEntity(  
        "http://localhost:8000/currency-exchange/from/{from}/to/{to}",  
        CurrencyConversion.class, uriVariables);
```

Web Client

- Od Spring 5, alternativa REST Template-u, uz podršku za **asinhronu pozivu**.
- Kreiranje na osnovu URL-a: `WebClient.create("http://localhost:8080")`.
- Kreiranje customizovane varijante preko buildera (`baseUrl`), (`defaultCookie`), (`defaultHeader`), (`defaultUriVariables`)...

Rest Client

- Od Spring 6.1 – slična infrastruktura kao RestTemplate, API sličan WebClient-u.
- Potrebno je kreirati bean:

```
java

@Bean
public RestClient restClient() {
    return RestClient.builder().baseUrl("http://localhost:8080").build();
}
```

- Poziv:

```
java

@GetMapping("/test/message")
public Echo echo(@PathVariable String message) {
    return restClient.get().uri("/test/" + message).retrieve().body(Echo.class)
}
```

Open Feign

- Potreban je jednostavniji i efikasniji način za pozivanje drugih mikroservisa.
- REST Template i WebClient zahtevaju više koda uz eksplicitno specificiranje URL-a servisa.
- **Deklarativni REST klijent** – kreira dinamičku implementaciju interfejsa deklarisanog kao Feign Client.
- Dobro se integriše sa drugim bibliotekama važnim za mikroservise.

Podešavanje:

- Zavisnost: `spring-cloud-starter-openfeign`.
- Glavna klasa klijentske aplikacije: `@EnableFeignClients`.
- Kreira se **Proxy interfejs** sa anotacijom `@FeignClient(name="currency-exchange", url="localhost:8000")`.
- U interfejs se dodaju zaglavlja metoda identična onima iz REST kontrolera.

- Autowire-ovanjem Proxy-ja pozivaju se njegove metode, koje se preusmeravaju na pozive na zadatoj lokaciji.

Middleware infrastrukture

- Sinhrono povezivanje putem HTTP-a dovodi do jake povezanosti i problema skalabilnosti.
- Rešenje: **asinhrona razmena poruka** upotrebom message broker-a – implementira **event-driven** arhitekturu.
- Događaji ukazuju da se nešto desilo u sistemu; publish-uju se u kanal za poruke od strane poslovne logike.
- Komponente zainteresovane za tip događaja se **subscribe**-uju na kanal.
- Događaji su vezani za **publish-subscribe** pattern, povezani sa message broker-om / message bus-om.
- Message broker omogućuje komunikaciju prevođenjem poruka između formalnih messaging protokola.
- **Producer** šalje poruke brokeru, **consumer** ih konzumira – publisher i konzumer se razdvajaju, arhitektura je horizontalno skalabilna.

Primeri middleware infrastrukture:

- **RabbitMQ** – distribuirani message broker, sakuplja streaming podatke iz više izvora i rutira na destinacije radi procesiranja.
 - **Apache Kafka** – streaming platforma za real-time pipeline-ove podataka, visoko skalabilan sistem poruka, otporan na greške.
-

14. Naming server

- Ako želimo dodati nove instance nekog REST servisa, moramo ih eksplicitno navesti na klijentskoj strani – loš pristup zbog hard-coded adresa; potrebno je dinamičko dodavanje/uklanjanje instanci.

Šta je naming server

- Obezbeđuje automatsku detekciju servisa na mreži – mikroservisi mogu da lociraju jedni druge i komuniciraju.
- Svaki mikroservis je standalone aplikacija sa specifičnom funkcionalnošću; da bi funkcionisali kao celina, moraju znati lokacije jedni drugih.
- **Service discovery** – održava zapis lokacija servisa, kako bi omogućio da se pronađu i komuniciraju.

Eureka

- **Spring Cloud Eureka** je deo Spring Cloud Netflix projekta.
- Predstavlja **registar servisa** sa 2 važne funkcionalnosti:
 1. Omogućuje mikroservisima da se registruju
 2. Omogućuje mikroservisima da pronađu druge servise
- Ponaša se kao "telefonski imenik" za mikroservise – mehanizam za registraciju i pronalaženje (service-to-service discovery).

Podešavanje Eureka:

- Kreira se Spring Boot aplikacija sa zavisnošću za Eureka server i anotacijom `@EnableEurekaServer` nad glavnim klasom.
 - U mikroservise se dodaje zavisnost za **eureka-client**.
 - Ako je port podešen na npr. 8761, na adresi `localhost:8761` vidi se spisak registrovanih servisa.
-

15. Load balancing

- **Load balancing** – distribuiranje saobraćaja među različitim instancama iste aplikacije.
- Da bi sistem bio otporan na greške, potrebno je da postoji više instanci jedne aplikacije.
- Strategije zavise od načina i nivoa distribuiranosti.

Dva tipa load balancing-a

Client-side load balancing

- Klijent "odlučuje" koju instancu će pozvati.
- Instance mikroservisa mogu biti deploy-ovane na različitim serverima.
- Logika load balancing-a je na klijentu, koji je svestan liste servera i na osnovu algoritma bira server.
- Primeri: **Spring Cloud Load Balancer, Netflix Ribbon**.

Server-side load balancing

- Instance su deploy-ovane na raznim serverima, a load balancer se postavlja "ispred" njih.
- Svi zahtevi prvo idu na load balancer, koji odlučuje kojoj instanci prosleđuje poziv.
- Nedostatak: **single point of failure**, povećana latentnost mreže.
- Primeri: **Spring Cloud Gateway, Netflix Zuul**.

Implementacija (Eureka + Feign)

- Eureka klijent u kombinaciji sa Feign-om podrazumeva podršku za Spring Cloud Load Balancer (client-side pristup).
- Dovoljno je navesti ime aplikacije, bez porta:

```
java

// Umesto: @FeignClient(name="currency-exchange", url="localhost:8000")
@FeignClient(name = "currency-exchange")
public interface CurrencyExchangeProxy { ... }
```

Spring Cloud Load Balancer

- Sloj između servera na kojima su deploy-ovani mikroservisi i klijenata.
 - Distribuirao ulazni saobraćaj na servere, smanjujući opterećenje.
 - Može koristiti različite algoritme (npr. round-robin).
-

16. Spring Cloud Gateway

API Gateway

- Predstavlja **ulaznu tačku** za pozive API-ja od strane klijenata ka servisima.
- Vršio procesiranje poziva – autentifikacija, rutiranje i sl.
- Olakšava komunikaciju između klijenta i servera.

Spring Cloud Gateway

- Obezbeđuje jednostavan, ali efikasan pristup za rutiranje ka API-jima; upravljanje zajedničkim osobinama mikroservisa (sigurnost, monitoring, resiliency).
- Klijent treba samo da zna kako da pristupi gateway-ju; upravljanje otkazivanjem servisa ili izmenama se dešava na gateway-u, odvojeno od klijenta.
- Postao je **default-ni API gateway** unutar Spring zajednice.
- Bolja propusnost od alternativa (posebno pri velikom broju konkurentnih korisnika), dobra integracija sa drugim bibliotekama (za razliku od **Netflix Zuul**-a, nekada popularnog izbora).

Kreiranje:

- Klasičan Spring Boot projekat sa odgovarajućom zavisnošću za Spring Cloud Gateway.
- Potrebno je da bude Eureka klijent.
- Enable-ovanje poziva kroz gateway ka servisima:


```
properties
```

```
spring.cloud.gateway.discovery.locator.enabled=true
```

Custom rute

- Moguće je kreirati custom-izovane rute ka servisima – rute su sredstvo preko kojeg klijenti pozivaju odgovarajuće servise.
- Moguće je izvršiti preusmeravanje, filtriranje, dodavanje zaglavlja/parametara.

Globalni filteri

- Kada svaki zahtev treba da prođe kroz filter – kreira se klasa (komponenta) koja implementira interfejs `GlobalFilter` i implementira metodu `filter`.
- Korisno npr. za dodavanje Logger-a koji loguje informacije o svakom zahtevu.

17. Resilience4j

Circuit breaker – motivacija

- Pri radu sa mikroservisima postoji dugačak lanac poziva: Mikro servis1 → Mikro servis2 → ... → Mikro servisN.
- Šta ako jedan od servisa prestane raditi ili radi jako sporo? Utiče na ceo lanac poziva.

Ključna pitanja:

- Možemo li vratiti **fallback response** ako servis ne radi? (moguće za neke slučajeve, npr. defaultni podaci; ne za bankovne transakcije)
- Možemo li implementirati **circuit breaker pattern** da smanjimo opterećenje – prekinuti pozivanje problematičnog servisa?
- Možemo li ponoviti isti request ako je greška privremena (**retry**)?
- Možemo li implementirati **rate limiting** – ograničiti broj poziva u periodu?

Rešenje: **Resilience4j** – lightweight biblioteka za implementaciju fault-tolerant osobina. U ranijim verzijama se najviše koristio **Hystrix**.

Zavisnosti: `starter-actuator`, `starter-aop`, `resilience4j-spring-boot2`.

Retry

```
java
```

```
@Retry(name = "sample-api")
```

- Podrazumevano: metoda se pokušava pozvati 3 puta ukupno pre nego što se prijavi greška.

properties

```
resilience4j.retry.instances.sample-api.maxRetryAttempts=5
resilience4j.retry.instances.sample-api.waitDuration=5
resilience4j.retry.instances.sample-api.enableExponentialBackoff=true
```

- Fallback metoda:

java

```
@Retry(name = "sample-api", fallbackMethod = "anotherMethodName")
```

Fallback metoda mora imati parametar tipa `Throwable` (Exception nasleđuje Throwable).

Circuit breaker

java

```
@CircuitBreaker(name = "default", fallbackMethod = "anotherMethodName")
```

- Predstavlja se kao konačan automat sa **3 normalna stanja**: `CLOSED`, `OPEN`, `HALF_OPEN`, i **2 specijalna stanja**: `DISABLED`, `FORCED_OPEN`.
- Dve opcije agregacije ishoda poziva:
 - **count-based sliding window** - agregira ishod poslednjih N poziva
 - **time-based sliding window** - agregira ishod u jedinici vremena
- U stanju **OPEN**, circuit breaker odbija sve nove pozive sa `CallNotPermittedException`. U ovo stanje dolazi kada se desi dovoljno poziva sa greškom.
- Nakon isteka vremena čekanja, prelazi u **HALF_OPEN** - dozvoljava određen broj poziva da se utvrdi da li je greška još prisutna.
- Ako se broj poziva sa greškom smanji ispod zadatog opsega → vraća se u **CLOSED**.
- Veoma je konfigurabilan.

Rate limiter

java

```
@RateLimiter(name = "default")
```

properties

```
resilience4j.ratelimiter.instances.default.limitForPeriod=1000  
resilience4j.ratelimiter.instances.default.limitRefreshPeriod=10s
```

Bulkhead (ograničenje konkurentnih poziva)

```
java  
  
@Bulkhead(name = "default")
```

```
properties  
  
resilience4j.bulkhead.instances.default.maxConcurrentCalls=100
```

18. Docker – osnovni koncepti

Uvod – motivacija

- Poslovne aplikacije sve više teže mikroservisnoj arhitekturi – mali, fokusirani servisi, razvijeni u proizvoljnom jeziku/framework-u.
- Deploy-ovanje mikroservisa je izazovno, posebno u različitim tehnologijama; potreban je jedinstven, univerzalan način za deploy-ovanje.
- **Kontejneri** rešavaju ovaj problem – za svaki mikroservis se kreira **docker image**, koji sadrži sve što je servisu potrebno za pokretanje.

Docker je open-source projekat pokrenut 2013. godine, inicijalno za Linux, sada radi i pod Windows-om i macOS-om. Podržava cloud-native razvoj.

Dva značajna koncepta:

- **Virtuelizacija** – uvođenje Guest OS-a nad postojećim Host OS-om, omogućava pokretanje OS-a na virtuelnim mašinama bez novih hardverskih resursa.
- **Kontejnerizacija** – forma virtuelizacije OS-a gde se aplikacije pokreću unutar izdvojenih celina (kontejnera), koristeći isti deljeni operativni sistem.

Docker olakšava kreiranje, deploy-ovanje i pokretanje aplikacija pomoću kontejnera; omogućava pakovanje aplikacija sa svim komponentama kao jedne celine. Kontejneri su lightweight i mogu se izvršavati direktno na host mašini – na jednom hardveru može raditi više kontejnera. Docker obezbeđuje "labavo izolovano" okruženje, bezbedno za istovremeno pokretanje više kontejnera. Docker fajlovi se lako repliciraju na novi hardver, olakšava se rollback na prethodnu verziju.

Docker image sadrži:

- Application runtime (JDK, Python, NodeJS)

- Kod aplikacije (arhiva, kompajliran kod)
- Zavisnosti (potrebne biblioteke)

Docker kontejneri se pokreću na isti način na svim infrastrukturama (lokalna mašina, data centar, cloud) – zbog toga su mikroservisi i Docker sjajan spoj.

Zašto je Docker popularan

- Može se instalirati i na cloud-u; većina cloud providera nudi servise bazirane na kontejnerima.
- Problem VM arhitekture – "heavyweight" (postoje host i guest OS).
- Docker prevazilazi ovo – na host OS-u treba da postoji samo Docker engine.

19. Docker – arhitektura, koncepti

Osnovni koncepti Docker-a

Pojam	Opis
Dockerfile	tekstualni fajl sa instrukcijama za build docker image-a; specificira OS, jezike, environment varijable, lokacije fajlova, mrežne portove itd. <code>docker build</code> procesuirala linije sekvencijalno
Docker image	sadrži izvršni kod i sve što je potrebno za pokretanje kao kontejner; ima više slojeva (svaki potiče od prethodnog, ponovno iskoristivi, ubrzavaju build); statička komponenta, niz bajtova; analogno snapshot-u kod VM
Docker container	lightweight, standalone, izvršiv softverski paket; izolovano okruženje za izvršavanje koda; nije svestan operativnog sistema ispod; dinamički koncept – radna verzija image-a, moguće više instanci istog image-a
Docker hub	javni repozitorijum image-a (najveća biblioteka/zajednica), preko 100.000 image-a; <code>hub.docker.com</code>
Docker daemon	background service na hostu, upravlja build-ovanjem, pokretanjem i distribucijom kontejnera; posrednik između engine-a i klijenta
Docker engine	klijent-server aplikacija koja kreira i pokreće kontejnere; kreira daemon proces na strani servera
Docker registry	mesto gde se čuvaju image-i (lokalni ili javni, npr. Docker Hub); organizovan u repozitorijume
Docker repository	omogućuje deljenje image-a sa timom/zajednicom; sadrži image-e kao tag-ove; <code>push</code> dodaje nove image-e

- Ograničenje memorije: `(-m 512m)`
 - Ograničenje CPU-a: `(--cpu-quota=5000)` (100000 = 100%)
 - `(docker run)` je skraćenica za `(docker container run)`.
-

20. Distributed tracing

Motivacija

- Pri radu sa mikroservisima često postoje kompleksni lanci poziva među servisima.
- Pitanje: kako debug-ovati kada se pojave problemi? Kako ispratiti request kroz lanac mikroservisa?
- Rešenje: **Distributed tracing**.

Arhitektura

- Informacije sa mikroservisa se šalju **distributed tracing serveru**.
- Server nudi interfejs preko kog se prati šta se dešava sa zahtevima; komunicira sa bazom podataka gde čuva bitne informacije.
- Koristi se **Zipkin** za distributed tracing preko Docker-a.

Pokretanje Zipkin-a u Docker-u:

```
bash  
  
docker run -p 9411:9411 openzipkin/zipkin:2.23
```

Interfejs dostupan na `(localhost:9411)`. Potrebno je vezati mikroservise za njega.

Monitoring vs Observability

- Monitoring je **reaktivan**, observability je **proaktivan**; monitoring je podskup observability-ja.
- Observability – koliko dobro razumemo šta se dešava u sistemu. Koraci:
 - Prikupljanje podataka: metrike, logovi, trace-ovi
 - Dobijanje znanja: AI/operacije za detekciju nepravilnosti

OpenTelemetry

- Kolekcija alata, API-ja, SDK-ova za podešavanje, prikupljanje, generisanje i eksport telemetry podataka (metrike, logovi, trace-ovi).
- Jedinstven standard – nema potrebe za odvojenim standardima za metrike/logove/tracing.
- Gotovo sve cloud platforme danas nude podršku za OpenTelemetry.

Praktična konfiguracija u Spring Boot-u

Na nivou mikroservisa koriste se **Micrometer**, **OpenTelemetry** i **Zipkin**:

```
io.micrometer      -> micrometer-observation
io.micrometer      -> micrometer-tracing-bridge-otel
io.opentelemetry    -> opentelemetry-exporter-zipkin
```

Ako se koristi Feign, dodaje se i:

```
io.github.openfeign -> feign-micrometer
```

Konfiguracija procenta zahteva koji se sample-uju i logging pattern:

```
properties

management.tracing.sampling.probability=1.0
logging.pattern.level=%5p [${spring.application.name:}, %X{traceId:-}, %X{spanI
```

Kreiranje Docker image-a iz Spring Boot aplikacije

- Koristi se Maven plugin već dodat u projekat, uz konfiguraciju:

```
xml

<configuration>
  <image>
    <name>example/hello-${project.artifactId}:${project.version}</name>
    <pullPolicy>IF_NOT_PRESENT</pullPolicy>
  </image>
</configuration>
```

- Pokretanje kao Maven build, goal: `spring-boot:build-image -DskipTests`.
- Nakon toga pokretanje preko `docker run` komande.

Docker Compose

- Za pokretanje višestrukih kontejnera zajedno.
- Verzija: `docker-compose --version`.
- Konfiguracija preko YAML fajla.

Primer `docker-compose.yml`:

```
yaml
```

```
version: '3.7'
services:
  currency-exchange:
    image: example/currency-exchange:0.0.1-SNAPSHOT
    mem-limit: 700m
    ports:
      - "8000:8000"
    networks:
      - "currency-network"
    depends_on:
      - naming-server
```

Pokretanje: `docker-compose up`.

21. Container orchestration, Kubernetes

Container orchestration - motivacija

- Kada mikroservisi zažive u produkciji, potrebno je voditi računa o orkestraciji: željeni broj instanci svakog kontejnera, da rade optimalno.

Potrebne osobine:

- **auto-scaling** - skaliranje kontejnera po potrebama
- **service discovery** - omogućavanje mikroservisima da se pronađu
- **load balancer** - raspodela opterećenja među instancama
- **self healing** - provera stanja instanci i zamena problematičnih
- **zero downtime deployments** - uvođenje novih verzija bez prekida rada

Popularni alati za orkestraciju:

- AWS-specifičan: **AWS Elastic Container Service (ECS)**, **AWS Fargate** (serverless verzija)
- Cloud neutral: **Kubernetes** - može se pokrenuti praktično bilo gde:
 - AWS - Elastic Kubernetes Service (**EKS**)
 - Azure - Azure Kubernetes Service (**AKS**)
 - GCP - Google Kubernetes Engine (**GKE**)

Kubernetes

Zašto Docker nije dovoljan?

- Docker image se može pokrenuti na bilo kojoj platformi (`docker run`), bez pitanja o jeziku, framework-u, OS-u ili konfiguraciji.

- Problem: kako obezbediti veći broj različitih instanci kada je povećano opterećenje i kada imamo veći broj aplikacija?

Kubernetes cluster – grupa servera koji se zajedno managuju. Deploy-ovanjem kroz Kubernetes moguće je pokrenuti željeni broj instanci, uz automatsku podršku za load balancing.

- Ako neka instanca prestane da radi, ostale nastavljaju rad, a zaustavljena se nadoknađuje novom (self-healing).
- Broj instanci se može automatski podešavati po potrebi (auto-scaling).
- Uvođenje nove verzije se dešava nepremitno (bez prekida rada).
- Sve pogodnosti se postižu jednostavnim komandama.

Google Kubernetes Engine (GKE)

- Kreiranje i održavanje Kubernetes klastera nije jednostavan zadatak – GKE nudi rešenje.
- Ista platforma koja se koristi za YouTube, Google Maps, Google pretragu.
- Potreban je Google Cloud free trial nalog (<https://cloud.google.com>) – kredit od 300\$, važi 12 meseci.

Kubernetes je resource manager – upravlja resursima (virtuelnim serverima) u cloud-u. Različiti provideri imaju različite nazive:

Provider	Naziv resursa
Amazon	EC2 (Elastic Compute Cloud)
Azure	Virtuelne mašine
Google Cloud	Compute Engines
Kubernetes	Nodes (čvorovi)

Dve vrste čvorova: master čvorovi i worker čvorovi.

Klaster predstavlja kolekciju čvorova. Kreira se jednostavno (Kubernetes Engine → Create cluster); najbolja praksa – brisati klaster posle svake upotrebe (naplata resursa).

Fun facts: K8S – skraćenica za Kubernetes (izgovara se "KOO-BER-NET-EEZ"); logo u obliku kormilara – Kubernetes na grčkom znači kormilar broda.

Nakon konektovanja na klaster (preko Google Cloud Shell), koristi se **kubectl** (Kube Controller) za deploy-ovanje aplikacije na osnovu docker image-a.

22. Kubernetes – osnovni koncepti, arhitektura

Osnovni koncepti (Single Responsibility princip – jedan koncept, jedna odgovornost)

Pods

- Najmanja jedinica koja se može deploy-ovati unutar Kubernetes-a.
- Kontejneri žive unutar pod-ova; unutar jednog pod-a može biti više kontejnera koji dele resurse i komuniciraju kroz localhost.
- Svaki pod ima jedinstvenu IP adresu, namespace (razdvajanje delova klastera), labela i anotacije (meta informacije).
- Pod je omotač za skup kontejnera.

Replica set

- Osigurava da se određen broj podova izvršava u svakom momentu.
- Ako se pod obriše, a definisan je jedan pod, replica set odmah pravi novi – neprestano prati pod-ove.
- Tipično se vezuje za jednu verziju aplikacije.

Deployment

- Scenario: imamo 3 instance aplikacije, pokušamo zameniti image za novu verziju, ali image ne postoji → stare instance nastavljaju rad, nova prijavljuje grešku.
- Imamo 2 replica seta – jedan za staru, jedan za novu verziju; njih kreira **deployment**.
- Kada je nova verzija ispravna, deployment postepeno ukida pod-ove stare verzije uz kreiranje pod-ova nove verzije (rolling update).

Servisi (Services)

- Iako svaki pod ima svoju IP adresu, aplikacija je dostupna na jedinstvenom URL-u sa load-balance-ovanim podovima (pod je throwaway unit) – zahvaljujući servisima.
- Servis omogućuje prihvatanje pristižućeg sadržaja kroz trajnu IP adresu.
- Automatski se dodeljuje **Load balancer** (primer servisa).

Kubernetes arhitektura

Master čvor – komponente:

- **etcd** – distribuirana baza podataka; svi detalji izvršenih komandi (stanja) čuvaju se ovde.
- **API server** – prihvata i procesira komande.
- **Scheduler** – raspoređuje pod-ove po čvorovima.
- **Controller manager** – osigurava da je trenutno stanje jednako željenom stanju.

Worker čvor – komponente:

- **Node agent – Kubelet** – prati dešavanja na čvoru i komunicira ih ka master čvoru (controller manager-u).
- **Networking component** – expose-ovanje deployment-a kao servisa.
- **Container runtime** – za pokretanje kontejnera, najčešće Docker.
- **Pods** – podovi koji se izvršavaju na čvoru.

Ako master čvor prestane raditi, worker čvor i dalje radi (aplikacija radi), ali nije moguće izvršiti nikakve izmene.

Praktični koncepti (deploy na Kubernetes)

Instalacija alata: **gcloud** (CLI za Google Cloud) i **kubectl**.

Prelaskom na Kubernetes, iz zavisnosti se mogu isključiti: Eureka naming server, Config server, Zipkin tracing – Kubernetes nudi gotova rešenja za ove stavke, uključujući svoj **service discovery**.

Health check (readiness/liveness) – property vrednosti (default-ne):

properties

`management.endpoint.health.probes.enabled=true`

`management.health.livenessState.enabled=true`

`management.health.readinessState.enabled=true`

Potrebno je uključiti **actuator** kao zavisnost.

Povezivanje preko Feign-a sada podrazumeva hard-code-ovanje adrese u obliku:

```
url="${CURRENCY_EXCHANGE_SERVICE_HOST:http://localhost}:8000"
```

`CURRENCY_EXCHANGE_SERVICE_HOST` je environment varijabla; ako nije dostupna, koristi se `localhost`.

Deploy-ovanje na Kubernetes:

bash

```
kubectl create deployment currency-exchange --image=rzk/currency-exchange-servi
```

```
kubectl expose deployment currency-exchange --type=LoadBalancer --port=8000
```

```
kubectl get service      # ili: kubectl get svc
```

Environment varijable

- Za svaki kreirani pod, Kubernetes automatski kreira environment varijablu (npr. za expose-ovan servis `CURRENCY_CONVERSION` → `CURRENCY_CONVERSION_SERVICE_HOST`).

- Preporučljivo je koristiti custom environment varijable, jer se default-ne kreiraju samo kada je dati servis dostupan.

```
yaml
env:
  - name: CURRENCY_EXCHANGE_URI
    value: http://currency-exchange
```

Declarative configuration – putem YAML fajla:

```
bash

kubectl get deployment currency-exchange -o yaml > deployment.yaml
kubectl get service currency-exchange -o yaml > service.yaml
kubectl apply -f deploymentandservice.yaml
```

Service discovery i load balancing su automatski uključeni od strane Kubernetes-a. Enable-ovanje API-ja (npr. Cloud Logging API, Stackdriver API za monitoring) – kroz **API & Services**.

Centralizovana konfiguracija

```
bash

kubectl create configmap currency-conversion \
  --from-literal=CURRENCY_EXCHANGE_URI=http://currency-exchange
```

```
yaml
envFrom:
  - configMapRef:
      name: currency_conversion
```

Centralizovani logging – opcije *View logs* i *View GKE Dashboard*.

Liveness i readiness probes

- Kubernetes probes za health check mikroservisa.
- Ako **readiness** probe nije uspešna – ne šalje se saobraćaj mikroservisu.
- Ako **liveness** probe nije uspešna – pod se restartuje.
- Spring Boot Actuator (≥2.3) obezbeđuje ugrađene mehanizme: `/health/readiness`, `/health/liveness`.

```
yaml
```

```
readinessProbe:
  httpGet:
    port: 8000
    path: /actuator/health/readiness
livenessProbe:
  httpGet:
    port: 8000
    path: /actuator/health/liveness
```

Autoscaling

```
bash
```

```
kubectl autoscale deployment currency-exchange --min=1 --max=3 --cpu-percent=5
```

YAML

- YAML **nije** markup jezik – user-friendly jezik za serijalizaciju podataka za sve programske jezike.
- Najčešće se koristi za konfiguracione fajlove, prenos i skladištenje podataka.
- Prvi put se pojavljuje 2001. godine; originalno je značio "Yet Another Markup Language".
- Sintaksa srodna JSON-u, koristi indentation nalik na Python.

Opseg (granularnost) mikroservisa

- Prva asocijacija: mikroservis je nešto veoma malo – tačno u odnosu na monolit, ali tendencija ka "veoma sitnim" mikroservisima je problematična.

Primer – bankarski sistem (računi i korisnici):

- Jedan mikroservis obuhvata osnovne operacije nad računom (otvaranje, zatvaranje, isplata, uplata) preko jednog REST interfejsa.
- Potreba: prebacivanje novca sa jednog računa na drugi.

Pristup 1 – zaseban mikroservis (nov REST interfejs) → **problem**: 2 poziva (skidanje i dodavanje sredstava) → **two-phase commit problem** (ako prva operacija uspe, a druga ne).

Pristup 2 – drugi mikroservis pristupa istoj bazi → rešava prethodni problem, ali stvara novi: **"coupling at the database"** – narušava princip da svaki mikroservis poseduje svoje podatke.

Pristup 3 – sve u okviru jednog mikroservisa, sa 2 REST interfejsa.

Zaključak: potrebno je primeniti **domain driven design**, umesto fokusa na tehnologiju – podela na Data/Domain/User mikroservise nije optimalna. Umesto toga, fokus na **domene** –

vertikalna podela umesto horizontalne; arhitektura treba da se sastoji od nezavisnih servisa organizovanih po poslovnom domenu.

Donja granica opsega:

A Microservice should consist of no less than an Aggregate (or at least an independent Entity) and the associated Services that operate on the entities of that Aggregate.

- **Aggregate** – koncept domain driven dizajna; grupe povezanih entiteta čiji su životni ciklusi povezani, pa ih tretiramo kao jedinicu.

Pronalaženje pravog nivoa apstrakcije: Event storming – identifikovanje najvažnijih događaja u poslovnom domenu, njihovo slaganje u vremensku sekvencu, identifikovanje komandi koje pokreću te događaje, potrebnih podataka i polisa (logike koja povezuje sledeće događaje).

Gornja granica opsega:

A Microservice shall be no larger than that which allows a two-pizza team to release a single complete, appropriately sized user story to production within a single day.

Kraj skripte.